MEMORIA TÉCNICA DEL PROYECTO FIN DE GRADO (ASIR) - DESPLIEGUE, SECURIZACIÓN Y AUTOMATIZACIÓN DE INFRAESTRUCTURA CLOUD EN OCI.

1. INTRODUCCIÓN Y OBJETIVOS

1.1. Justificación del Proyecto: El presente proyecto documenta de forma exhaustiva la arquitectura, fortificación, despliegue de servicios y automatización de un servidor Linux alojado en Oracle Cloud Infrastructure (OCI) bajo una instancia de recursos ultra-optimizados (VM.Standard.E2.1.Micro con 1GB de RAM). El propósito es demostrar la capacidad técnica para diseñar, administrar y asegurar un entorno de producción real que alberga una infraestructura web optimizada, servicios de red privados mediante VPN y un pipeline de despliegue continuo adaptado a dispositivos heterogéneos.

1.2. Objetivos Principales:

- Aprovisionar una máquina virtual en la nube (OCI) adaptada a restricciones severas de hardware (1GB de RAM).
- Implementar ajustes avanzados a nivel de kernel, memoria SWAP y control de congestión TCP BBR.
- Aplicar una estrategia de seguridad perimetral y bastionado de sistemas (Hardening, UFW, Fail2Ban, llaves Ed25519 y supresión de servicios innecesarios como rpcbind).
- Configurar un servidor web multi-sitio (Nginx) con optimización WPO (compresión Gzip y caché estática), HTTPS automatizado mediante Certbot y directivas SEO avanzadas.
- Implementar una red privada virtual (WireGuard) con aislamiento multi-peer para el tráfico seguro.
- Desarrollar un pipeline CI/CD híbrido basado en Python 3 para automatizar la integración y el despliegue desde dispositivos móviles (iPadOS) y portátiles (macOS).
- Gestionar el control de versiones del proyecto mediante Git y GitHub con saneamiento de credenciales.

2. FASE 1: APROVISIONAMIENTO Y OPTIMIZACIÓN DEL SO (INFRAESTRUCTURA OCI)

- Instancia Cloud: Aprovisionamiento de máquina virtual Ubuntu Server en OCI con asignación de IP pública estática y direccionamiento en la VCN (Virtual Cloud Network).
- Memoria de Intercambio (SWAP): Creación y activación de un archivo swapfile de 2GB para mitigar la intervención crítica del proceso OOM Killer debido al límite físico de memoria RAM.
- Control de Congestión TCP BBR: Activación del algoritmo de control de congestión de Google en el kernel de Linux para maximizar el rendimiento del flujo de red.
- Control y Rotación de Logs (Journald): Limitación estricta de la retención de registros del sistema a 200MB en /etc/systemd/journald.conf para evitar la saturación del almacenamiento local.

3. FASE 2: HARDENING Y SEGURIDAD PERIMETRAL (DEFENSE IN DEPTH)

- Bastionado del Servicio SSH: Modificación de /etc/ssh/sshd_config deshabilitando de forma absoluta la autenticación por contraseña (PasswordAuthentication no) y forzando el uso exclusivo de criptografía asimétrica basada en pares de llaves Ed25519/RSA.
- Mitigación de Ataques de Fuerza Bruta: Despliegue y configuración del daemon Fail2Ban para la monitorización heurística de registros de acceso y aplicación de baneos automáticos en el firewall.
- Firewall Híbrido: Configuración por capas combinando Listas de Seguridad y NSGs en la consola web de Oracle (Capa Cloud) con políticas de denegación por defecto implementadas mediante UFW (Capa Sistema Operativo).
- Auditoría de Sockets y Reducción de Superficie: Validación del estado de los puertos de escucha mediante ss -tulnp y desactivación de servicios remotos innecesarios no requeridos en producción (como rpcbind en el puerto TCP/UDP 111).

4. FASE 3: NETWORKING Y TÚNEL VPN (WIREGUARD)

- Despliegue de Red Privada Virtual: Instalación y configuración de WireGuard operando directamente sobre el espacio del kernel de Linux, escuchando en el puerto UDP 51820 (previamente abierto en el firewall de OCI).
- Definición de Subred Lógica: Establecimiento del direccionamiento privado cifrado bajo el bloque CIDR 10.66.66.0/24.
- Entorno Multi-Peer: Configuración de perfiles de cliente aislados en /etc/wireguard/wg0.conf mediante enrutamiento por claves públicas (*Cryptokey Routing*), permitiendo el acceso concurrente y seguro de múltiples dispositivos (como estaciones de trabajo Mac y dispositivos móviles) sin colisiones en las tablas de encaminamiento.

5. FASE 4: ARQUITECTURA MULTI-SITIO, RENDIMIENTO WEB Y SEO (NGINX)

- Servidor Web y Proxy Inverso: Instalación de Nginx para gestionar el enrutamiento de peticiones HTTP/HTTPS.
- Server Blocks: Creación de bloques de servidor orientados a nombres de host para el dominio principal (robertmesa.es) y subdominios (www), integrando reglas de reescritura try_files para soportar aplicaciones web de tipo SPA (Single Page Application).
- Cifrado TLS/SSL Automatizado: Implementación de Certbot (Let's Encrypt) para la expedición de certificados digitales y redirección permanente (301) del tráfico inseguro desde el puerto 80 al puerto 443.
- Optimización del Rendimiento Web (WPO): Activación del módulo de compresión dinámica Gzip (gzip on, gzip_types) y establecimiento de políticas de caché de

navegador (Cache-Control, expires 1y) para recursos estáticos (JS, CSS, imágenes, SVG).

- Optimización SEO Avanzada: Inyección de metadatos estructurados en formato JSON-LD, configuración de directivas estrictas en robots.txt y estructuración de sitemaps para facilitar el rastreo y la indexación de motores de búsqueda como Googlebot.

6. FASE 5: PIPELINE CI/CD HÍBRIDO Y AUTOMATIZACIÓN DE DESPLIEGUES

Desarrollo de un flujo de integración continua autónomo basado en scripting de Python 3 ejecutado en el servidor (deploy_server.py) para actualizar entornos de producción sin dependencia de ordenadores de escritorio:

- Detección Dinámica: Monitorización heurística en la capa de usuario de paquetes comprimidos en formato .zip transferidos mediante canales SFTP seguros.
- Sandbox y Limpieza: Extracción del contenido en un directorio temporal aislado con purga automática de metadatos no deseados generados por otros sistemas operativos (como carpetas __MACOSX).
- Compilación Nativa: Ejecución síncrona de dependencias con Node.js v20 LTS y empaquetado de assets de producción mediante herramientas de compilación modernas (npm run build).
- Despliegue Atómico: Reemplazo síncrono y en caliente del directorio de producción (/var/www/danza) minimizando cualquier tiempo de inactividad (*downtime*).
- Garbage Collection: Limpieza automatizada y robusta de artefactos temporales mediante bloques de excepción estructurados (try/finally), garantizando la estabilidad del almacenamiento.